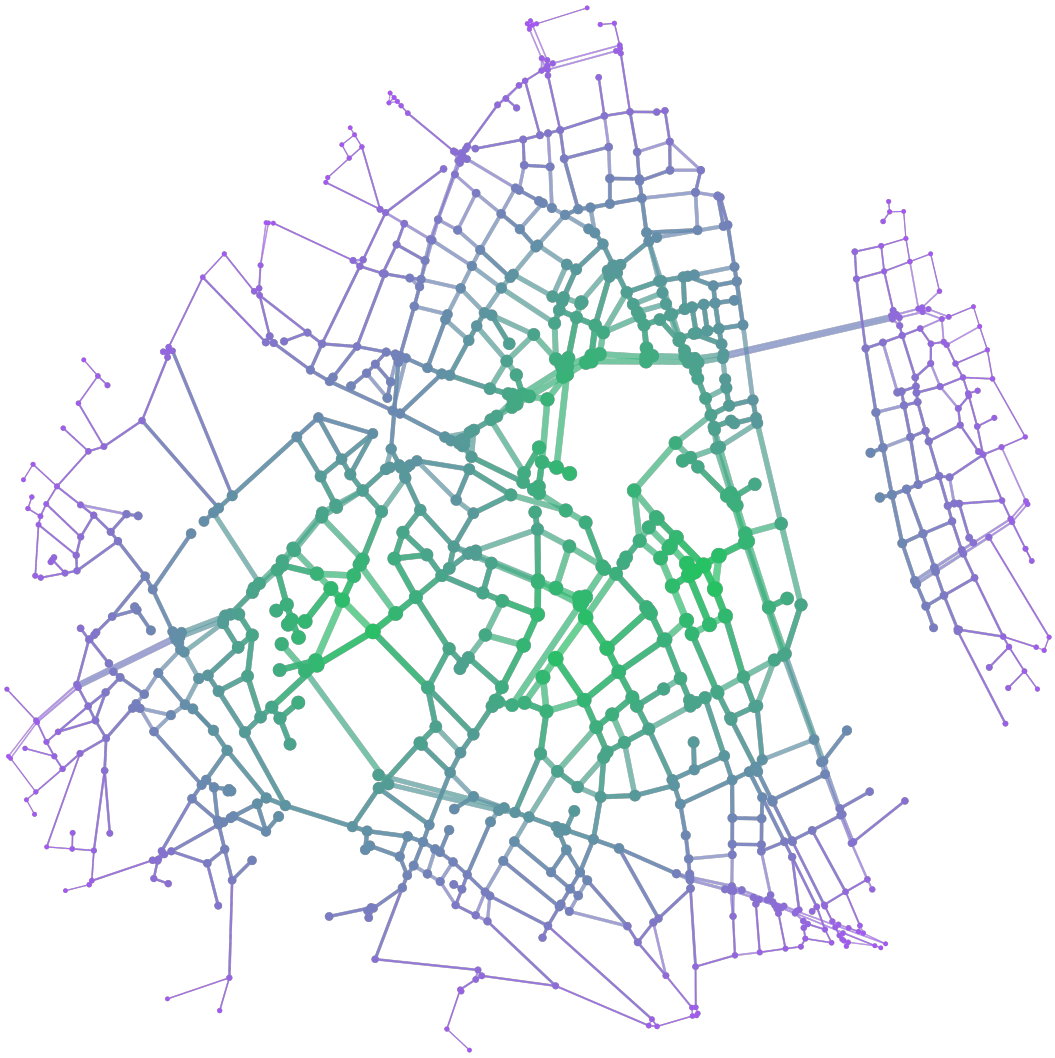
MEB

Mathematical Essays from Bonn



Bonn, 2026.1

Essays

a very long and nice title (Mathematical Essays Bonn) **3**

Strengths and Limits of Formalisation: CW complexes as an example (Hannah Scholz) **5**

a very long and nice title

Mathematical Essays Bonn

Hallo ich werde ein sample essay. Ich zitiere [1]. Darin steht nichts, denn es gibt dieses Buch gar nicht.
Hello, I am going to be a sample essay! Ich zitiere [1]. Nothing is written in there, because the book doesn't exist.

Bemerkung 1. Diese Bemerkung begrüßt dich auf Deutsch.

Remark 2. Diese Bemerkung begrüßt dich auf Englisch.

In der Bemerkung 2 wird man auf Englisch begrüßt. Unverständlich!
You can even link to other webpages! `www.google.com`

Theorem 3 (with a big name)
hello

Definition 4
hello

Beispiel 5. guten Tag! Ich bin ein Beispiel

Example 6. hello

Figure 1 is great!

Beweis. Das ist ein Beweis auf deutsch! □

Literatur

[1] MEB. *MEBs first book*. MEB, 2026.

MEB ist eine wunderbare Essaykollektive, um deinen mathematischen Gedanken freien Lauf zu lassen. MEB gibt es seit dem Wintersemester 2025/2026.

Proof. This is an english proof. The claim is correct! □

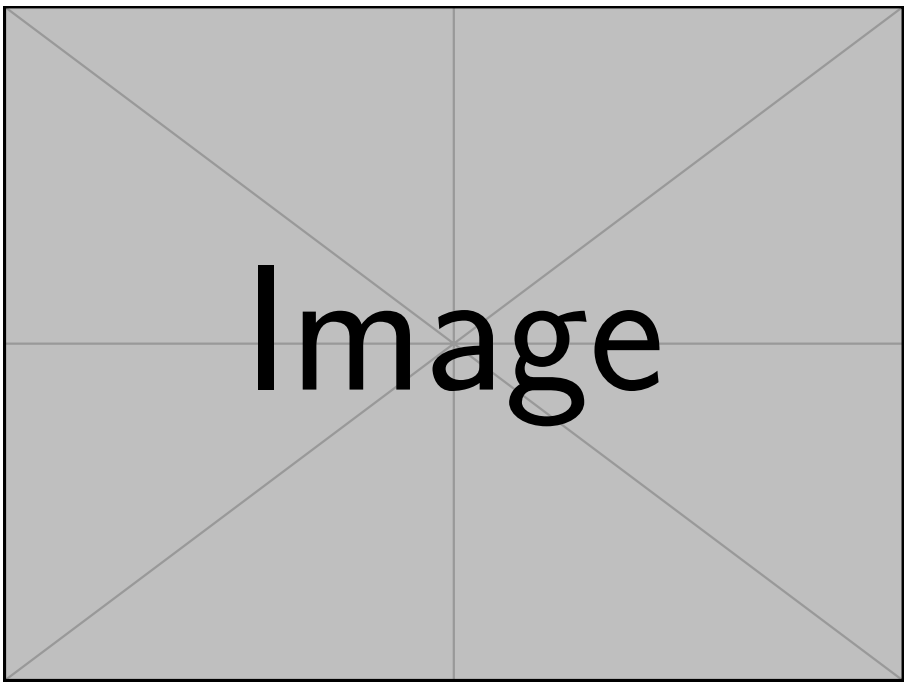


Abbildung 1: This is a figure.

Strengths and Limits of Formalisation: CW complexes as an example

Hannah Scholz

Introduction

Mathematical formalisation is an area of mathematics that has seen exponential growth in the last few years. In simple terms, formalisation is the practice of writing a proof in a programming language and then using the computer to verify the validity of the argument.
This works because of a correspondence between proofs and programs called the Curry-Howard isomorphism. Essentially, a proof that A implies B can be seen as a function that sends a proof of A to a proof of B. So when you write a proof of "A implies B" in a programming language, you are specifying such a function. Checking the validity of the proof then corresponds to checking that this function indeed sends proofs of A to proofs of B.
With the advent of powerful AI tools, formalisation has come to be seen as a magic tool that can lend absolute credibility to anyone using it. Every day we see new claims of solutions to previously unsolved problems supposedly backed by formalisation. Therefore, it is critical that we understand what formalisation can actually be helpful for and where its limits lie.
This is the focus of the following essay. I will explore this topic using, as an example, my formalisation of CW complexes in the language Lean that I did together with and supervised by Floris van Doorn. This text assumes no prior knowledge of formalisation or CW complexes. I will not explain basic terms from topology but the text should still be understandable without knowing them. This essay is heavily inspired by the essay "Why formalize mathematics?" written by Patrick Massot [4]. If this topic interests you, please check out that essay as well!

Defining CW complexes

CW complexes are an important class of spaces from the area of topology. Their importance is owed to the fact that they are very general spaces that at the same time allow for very powerful results. Intuitively, a CW complex is the result of glueing n-cells together along their edges where an n-cell is a continuous image of an n-disc. You can see examples of n-cells of different dimensions in Figure 1.
By gluing these cells together you can build a lot of the spaces that you already know. In Figure 2, you can see that intervals, the real line and spheres are all CW complexes.

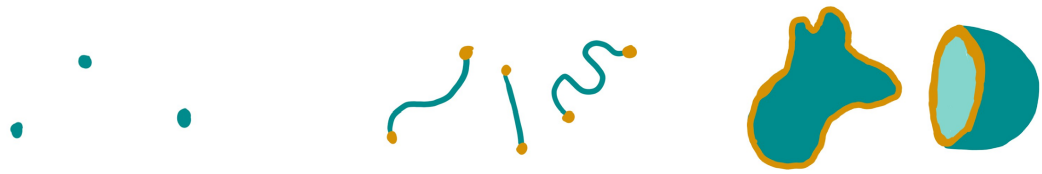


Figure 1: 0-, 1- and 2-cells.

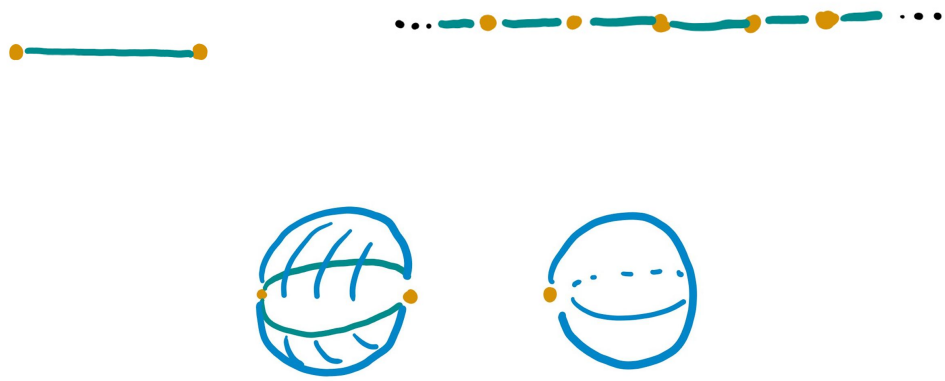


Figure 2: CW structures on the interval, real line and two structures on the 2-sphere.

Now, we should move from this intuition to the concrete definition. I know that it looks very long and scary but don't worry: I will offer an intuitive explanation below and you don't need to understand all of the details.

Definition 1

Let X be a Hausdorff space. An *(absolute) CW complex* on X consists of a family of indexing sets $(I_n)_{n\in\mathbb{N}}$ and a family of continuous maps $(Q_i^n\colon D^n \to X)_{n\in\mathbb{N},i\in I_n}$ called *characteristic maps* with the following properties:

(i) $Q_i^n|_{\mathrm{int}(D^n)} : \mathrm{int}(D^n) \to Q_i^n(\mathrm{int}(D^n))$ is a homeomorphism for every $n \in \mathbb{N}$ and $i \in I_n$. We call $e_i^n := Q_i^n(\mathrm{int}(D^n))$ an *(open) n-cell* and $\overline{e}_i^n := Q_i^n(D^n)$ a *closed n-cell*.

(ii) Two different open cells are disjoint.

(iii) For each $n \in \mathbb{N}$ and $i \in I_n$ the *cell frontier* $\partial e_i^n := Q_i^n(\partial D^n)$ is contained in the union of a finite number of closed cells of a lower dimension.

(iv) A set $A \subseteq X$ is closed if the intersections $A \cap \overline{e}_i^n$ are closed for all $n \in \mathbb{N}$ and $i \in I_n$.

(v) The union of all closed cells is X.

Here D^n refers to the closed unit n-disc. The convention for what the symbol D^n means is different in topology to other fields of mathematics (it is often used to refer to the open unit disc).

Now for a short explanation: Property (i) tells you that the open disk can be deformed in a reasonable way (e.g. no ripping or gluing). Note that the notation for the closed n-cell $\overline{e}_i^n$ is very confusing: it looks like the closed cell is the closure of the open cell but this is not the case in general. Property (iii) is called "closure finiteness" (which is the "C" in "CW complex"). It tells you that the cell frontier needs to be glued to cells of lower dimensions and that the new cell cannot stretch over infinitely many lower cells. Again, the notation for the cell frontier is confusing: it is not generally the frontier of the open cell. The "W" in "CW complex" stands for "weak topology". This is Property (iv). It tells you that the topology on X is determined by the cells. Lastly, Property (v) tells you that the whole space is made up of the cells.

Let us proceed to the Lean version of this definition which you can find in Figure 3. Again, this is very long and surely overwhelming if you do not know any Lean. I will try to offer an explanation below. You don't need to understand this line by line.

```
class CWComplex.{u} {X : Type u} [TopologicalSpace X] (C : Set X) where
  cell (n : ℕ) : Type u
  map (n : ℕ) (i : cell n) : PartialEquiv (Fin n → ℝ) X
  source_eq (n : ℕ) (i : cell n) : (map n i).source = ball 0 1
  continuousOn (n : ℕ) (i : cell n) : ContinuousOn (map n i) (closedBall 0 1)
  continuousOn_symm (n : ℕ) (i : cell n) :
    ContinuousOn (map n i).symm (map n i).target
  pairwiseDisjoint' :
    (univ : Set (Σ n, cell n)).PairwiseDisjoint
        (fun ni ↦ map ni.1 ni.2 '' ball 0 1)
  mapsTo' (n : ℕ) (i : cell n) : ∃ I : Π m, Finset (cell m),
    MapsTo (map n i) (sphere 0 1) (⋃ (m < n) (j ∈ I m),
        map m j '' closedBall 0 1)
  closed' (A : Set X) (hAC : A ⊆ C) :
    (∀ n j, IsClosed (A ∩ map n j '' closedBall 0 1)) → IsClosed A
  union' : ⋃ (n : ℕ) (j : cell n), map n j '' closedBall 0 1 = C
```

Figure 3: The definition of an absolute CW complex in Lean.

The first line says that we want to define the class of CW complexes `C` in a topological space `X`. Lean works with type theory instead of set theory, so you will see the word `Type` appear a lot in the code. Just mentally replace it with the word "Set". So, `{X : Type u}` means that `X` is a set.

`[TopologicalSpace X]` means that `X` is a topological space and `(C : Set X)` means that `C` is a subset of `X`. What the different brackets mean is not important here.
You can see that this is already different from Definition 1 since we are defining CW complexes as subspaces instead of the spaces themselves. The reason for this change is that a lot of CW complexes have a natural ambient space whose topology is easy to understand. For example, in Figure 2 I drew CW structures on the interval and the 2-sphere. These have the natural ambient spaces $\mathbb{R}$ and $\mathbb{R}^3$ respectively which have nicer topologies than the subspace topologies induced on our CW complexes. When we do maths on paper, this is a detail that we would just gloss over and ignore in most of our proofs. However, in formalisation we need to do all proofs in full detail and small issues like these can become painful. This leads us to our first limit:

> **Limit 1**
>
> Sometimes, a mathematical definition or statement needs to be modified to work well for formalisation.

Why is this a limit? This is because whatever statement about CW complexes I want to formalise, it will never be exactly the same as the original paper statement because we are working with a different definition. So the reader of my Lean code needs to check for themselves that my definition is still equivalent to the definition on paper.
Now, let's get back to our code in Figure 3. In line 2, we are defining the indexing sets `cell n`. These are just what we called I_n in Definition 1. In line 3, we define the characteristic maps `map n i` that correspond to Q_i^n from Definition 1. We define them as `PartialEquiv (Fin n →` $\mathbb{R}$`) X` where `Fin n →` $\mathbb{R}$ is just a weird way to write $\mathbb{R}^n$. A `PartialEquiv` is a little bit like a pair of inverse maps: it consists of two maps in opposite directions. In this case, we have the forward map called `map n i` going from $\mathbb{R}^n$ to `X` and the backward map called `(map n i).symm` going in the opposite direction. What makes this a *partial* equivalence is that these two maps are only inverses when restricted to certain sets in the domain and codomain. The relevant set in the domain, i.e. here $\mathbb{R}^n$, is called `(map n i).source` and the relevant set in the codomain, i.e. `X`, is called `(map n i).target`. For this definition to make sense, we require that the set in the codomain is the image of the set in the domain under the forward map. Why do we use such a weird definition? Again, constantly restricting to subsets is surprisingly annoying, so we prefer to have maps defined on our entire space. So this is another instance of Limit 1.
In line 4, we say that the specific set in $\mathbb{R}^n$ on which the maps are inverses is the open unit ball. This is just another way of saying that the restriction of `map n i` to the open unit ball is a bijection.
Then in line 5, we require `map n i` to be continuous on the closed unit ball and in lines 6 and 7, we require the backwards map `(map n i).symm` to be continuous on the set `(map n i).target` which in this case, as explained above, is just the image of the open unit ball under `map n i`. So

all together, lines 3 to 7 capture Property (i) of Definition 1. We can observe our first strength here:

> **Strength 1**
>
> Formalised definitions and notation are unambiguous.

Where before the notation D^n could mean different things to different mathematicians, `closedBall 0 1` is defined in the mathematical library "Mathlib" of Lean and by clicking on it in a code editor, you will get sent to the precise definition of `closedBall` which is

```
def closedBall (x : α) (ε : ℝ) := { y | dist y x ≤ ε }
```

i.e. all the points `y` that have distance to `x` less than or equal to ε. So even if the name was something unhelpful like `D` instead of `closedBall`, the meaning would still be clear.
For you to understand the next limit that I want to present, I first need to tell an embarrassing anecdote about line 4 of the Lean definition: for almost a year after I started working on this project (so in particular during my entire Bachelor thesis that covered it), this line mistakenly read

```
source_eq (n : ℕ) (i : cell n) : (map n i).source = closedBall 0 1
```

which requires the characteristic map to be a bijection on the entire closed ball. This in particular excludes examples like the second construction on the 2-sphere in Figure 2 where the cell frontier is mapped to a single point. I only figured this out when I tried to implement the examples shown in the figure and this particular example was not possible. The limitation here is an extension of Limit 1:

> **Limit 2**
>
> Formalisation cannot verify a definition, i.e. it cannot check that the formalised notion captures the paper definition.

Of course, the "wrong" definition that I wrote wasn't wrong in any mathematical sense. Like in paper mathematics, you are free to define whatever object you want in Lean. I had just accidentally defined a more restrictive version of CW complexes. This issue can happen to humans and it often happens to AIs. It can be used to make it seem like you have proven an amazing complicated statement when, in fact, three files deep a definition is translated incorrectly which makes the final statement trivial.
Sometimes, a mismatch between the paper and Lean definition is even intentional. You might have noticed that the Lean definition in Figure 3 does not require the space to be Hausdorff. This is because you can define the structure entirely without it and by not including it, users of my code have more freedom about the order in which they choose to prove things. However, this means that the correct way to state "X is a CW complex" in Lean is `[CWComplex X] [T2Space X]`

(where "T_2-space" is another name for Hausdorff spaces). This could lead to misunderstandings if you don't check the definitions carefully or read the documentation.
Let us finish discussing the Lean definition in Figure 3. The lines that we have not discussed are mostly just translations of our properties from Definition 1. Their precise statements are not particularly important so I will just tell you which statement they correspond to. Don't worry about understanding the code. Lines 8 to 10 correspond to Property (ii), lines 11 to 13 describe closure finiteness i.e. Property (iii), lines 14 and 15 describe the weak topology i.e. Property (iv) and the last line corresponds to Property (v).

Proving things about CW complexes

Next, I want to come to the most obvious strength of formalisation:

> **Strength 2**
>
> Formalisation can verify whether a proof is correct.

This can be useful for a number of reasons that I wish to discuss in this section. Firstly, this is very convenient for tedious proofs. For my thesis, I was proving a lot of very basic lemmas that had very annoying proofs. Before I can present an example, I need to give a definition first.

> **Definition 2**
>
> Let X be a CW complex in the sense of Definition 1. We define $X_n := \bigcup_{m<n+1} \bigcup_{i\in I_m} \overline{e}_i^m$ and call it the *n-skeleton* of X for $-1 \leq n \leq \infty$.

A simple but important fact is that a CW complex cannot just be expressed as the union of its closed cells but also as the union of its open cells. This fact is derived from the following lemma.

> **Lemma 3**
>
> $X_n = \bigcup_{m<n+1} \bigcup_{i\in I_m} e_i^m$ for every $-1 \leq n \leq \infty$.

I have put the proof of this lemma in Figure 4. Please don't try to actually go through it. This is just to illustrate how annoying these proofs are when you actually write out all the details. No one doubts that the statement of Lemma 3 is correct but (to my knowledge) no books write out proofs like these in excruciating detail either. So if you want a detailed proof, it is left to you to figure out and Lean can be of help here. I write proofs like these directly in Lean because I can think more clearly about my next step when I don't need to worry about the last one.
Of course, this is more of a toy example but Strength 2 has started to become useful in research mathematics: in 2020, Peter Scholze posed a challenge called the "Liquid Tensor Experiment" to the formalisation community [5]. He wanted a proof from his work about condensed mathematics verified because he still had slight doubts about its correctness. After the relevant proof had

We show this by induction over $-1 \leq n \leq \infty$. For the base case, assume that $n = -1$. Then we get $X_{-1} = \bigcup_{m<0} \bigcup_{i \in I_m} \overline{e}_i^m = \varnothing = \bigcup_{m<0} \bigcup_{i \in I_m} e_i^m$.
For the induction step, assume that that the statement is true for n. We now show that it also holds for $n+1$.

$$
\begin{aligned}
X_{n+1} &= \bigcup_{m<n+2} \bigcup_{i \in I_m} \overline{e}_i^m \\
&= \bigcup_{i \in I_{n+1}} \overline{e}_i^{n+1} \cup \bigcup_{m<n+1} \bigcup_{i \in I_m} \overline{e}_i^m \\
&= \bigcup_{i \in I_{n+1}} \overline{e}_i^{n+1} \cup X_n \\
&\overset{(1)}{=} \bigcup_{i \in I_{n+1}} \overline{e}_i^{n+1} \cup \bigcup_{m<n+1} \bigcup_{i \in I_m} e_i^m \\
&= \bigcup_{i \in I_{n+1}} e_i^{n+1} \cup \bigcup_{i \in I_{n+1}} \partial e_i^{n+1} \cup \bigcup_{m<n+1} \bigcup_{i \in I_m} e_i^m \\
&\overset{(2)}{=} \bigcup_{i \in I_{n+1}} e_i^{n+1} \cup \bigcup_{m<n+1} \bigcup_{i \in I_m} e_i^m \\
&= \bigcup_{m<n+2} \bigcup_{i \in I_m} e_i^m
\end{aligned}
$$

Note that (1) holds by induction and (2) holds by closure finiteness.
Now we can move on to the case $n = \infty$.

$$
\begin{aligned}
X_\infty &= \bigcup_{m<\infty+1} \bigcup_{i \in I_m} \overline{e}_i^m \\
&= \bigcup_{m<\infty+1} \bigcup_{l<m+1} \bigcup_{i \in I_l} \overline{e}_i^l \\
&= \bigcup_{m<\infty+1} X_m \\
&\overset{(1)}{=} \bigcup_{m<\infty+1} \bigcup_{l<m+1} \bigcup_{i \in I_l} e_i^l \\
&= \bigcup_{m<\infty+1} \bigcup_{i \in I_m} e_i^m
\end{aligned}
$$

Where (1) holds by induction.

Figure 4: The proof of Lemma 3.

been formalised in an effort led by Johan Commelin, he wrote another blog post [6] expressing his satisfaction and explaining that the formalisation deepened his understanding of the maths involved and led to simplifications. We can extract another strength here:

Strength 3

Formalising can clarify and aid understanding of the mathematics.

I have experienced this on a smaller scale with the proof of the main theorem in my thesis. This theorem says that the product of two CW complexes is a CW complex under certain conditions. To state this theorem we first need another definition:

Definition 4

Let X be a topological space.

(i) We call X *compactly coherent* if a set $A \subseteq X$ is open iff for all compact sets $C \subseteq X$, the intersection $A \cap C$ is open in C.

(ii) We can also turn an arbitrary X into a compactly coherent space by passing to a fine enough topology (the precise definition of which is irrelevant). We call the resulting space the *compact coherentification* of X.

The two definitions above cannot be found in the literature under these names. A "compactly coherent space" is usually called a "compactly generated space" or a "k-space" and the "compact coherentification" is typically referred to as the "k-ification". So why am I making up new terms here? This is a funny consequence of Strength 1. In the mathematical literature there are three

non-equivalent types of spaces that get called "compactly generated". To my knowledge, this distinction was first discovered by Wikipedia contributors in [7]. The three notions all agree on Hausdorff spaces which I assume is the reason no one had cared previously. One of the three notions had already been implemented in Lean as `CompactlyGeneratedSpace`, so I had to choose a different name (which was suggested by Steven Clontz).
Now we can state two results about the product of CW complexes:

Theorem 5

Let X and Y be CW complexes with the respective families of characteristic maps $(Q_i^n \colon D^n \to X)_{n\in\mathbb{N},i\in I_n}$ and $(P_j^m \colon D^m \to Y)_{m\in\mathbb{N},j\in J_m}$.

(i) Assume that $X \times Y$ is compactly coherent. Then $X \times Y$ is a CW complex with characteristic maps $(Q_i^n \times P_j^m \colon D^n \times D^m \to C \times E)_{n,m\in\mathbb{N},i\in I_n,j\in J_m}$ and indexing sets $K_l = \bigcup_{n+m=l} I_n \times J_m$.

(ii) In general, the compact coherentification of $X \times Y$ is a CW complex.

I am not going to give the proof but you can find a proof (which I would call more of a proof sketch) in [3]. I hope that you can nonetheless imagine how careful you need to be to actually work with the different topologies and to describe which finite set of cells each new cell attaches to. In fact, in a talk I held in my Bachelor thesis seminar, I explained that I wanted to formalise the proof of this theorem and went through the most interesting part of it. When I formalised it later, I realised that I had gotten confused with the topologies and that my argument was incorrect. Luckily, none of the attendees of that talk had noticed it either.
I believe that most students who read through delicate proofs like this one don't actually understand all these intricate details. I certainly did not. But of course this understanding doesn't come from knowing that a result has been formalised, it comes from doing this formalisation yourself or from stepping through the formal proof line by line. This is where we find another limit:

Limit 3

Formalisation cannot guarantee that proofs are "beautiful", reusable or even readable at all.

This is a big issue with AI-generated Lean code. Often, the person that generated it doesn't have the knowledge to or doesn't want to clean up the proofs to make them understandable to humans. This basically limits the use of such a formalisation to verifying whether a result is correct or not.
I want to end this section with the obvious question: have any big formalisation projects ever found the result they were formalising to be false? To my knowledge, the answer is no. Usually, some errors are discovered along the way but they seem to have always been fixable. This was

the case for example in the Liquid Tensor Experiment mentioned above. However, I think this is mostly because the results chosen for formalisation so far have either been standard results from university curricula or very famous theorems. In both of these cases, the results have been under high academic scrutiny and were therefore always likely to be correct. But I think it is only a question of time until a lesser known paper gets proven false through formalisation.

Conclusion

I hope this essay could give you a little bit of an overview of why people choose to formalise mathematics and of how the random person on the internet claiming that they have a formalised proof of an important unsolved problem was probably misled by their AI.
I should also leave you with probably the most important strength of formalisation:

> **Strength 4**
>
> Formalisation is fun!

In a Quanta article [2], Amelia Livingston is quoted saying

> You can do 14 hours a day in it and not get tired and feel kind of high the whole day. You're constantly getting positive reinforcement.

If this makes you want to try out Lean, I suggest you start by playing the Natural Numbers Game by Kevin Buzzard which can be found at `https://adam.math.hhu.de/`. If you want to read about how AI is currently affecting mathematics and formalisation I recommend you read the essay "Mathematicians in the Age of AI" by Jeremy Avigad [1]. If you want to read about how AI performances at the IMO can be interpreted, you can read the blog post "AI at IMO 2025: a round-up" by Kevin Buzzard. Lastly, if you want to hear more reasons for formalising maths, I want to repeat my recommendation from the introduction which is "Why formalize mathematics?" by Patrick Massot [4].

References

[1] Jeremy Avigad. Mathematicians in the age of AI, 2026.

[2] Kevin Hartnett. Building the mathematical library of the future. *Quanta Magazine*. Accessed: 2026-03-25.

[3] Allen Hatcher. *Algebraic topology*. Cambridge Univ. Press, 13. print. edition, 2010.

[4] Patrick Massot. Why formalize mathematics?, 2021.

[5] Peter Scholze. Liquid tensor experiment. Xena, December 2020. Accessed: 2026-03-24.

[6] Peter Scholze. Half a year of the liquid tensor experiment: Amazing developments. Xena, June 2021. Accessed: 2026-03-24.

[7] Wikipedia contributors. Compactly generated space — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Compactly_generated_space&oldid=1334561412, 2026. Accessed: 2026-03-25.

Hannah Scholz is a second year Master's student in Bonn. She wrote her Bachelor's thesis about formalising CW complexes in the proof assistant Lean which she has loved ever since. You can find out more about her on her webpage at https://scholzhannah.de.

Contributors

Authors

Axioml, Testautor Bert, Testautor Ernie

Editors

LaTeXand Design

You?

If you are a math person from Bonn and would like to help with the next edition of MEP, you can write us an e-mail at gpohlenz@uni-bonn.de. If you want to be added to our WhatsApp community, include your phone number. To get started with writing your essay, clone our repository https://github.com/MEB-Collective/MEB.